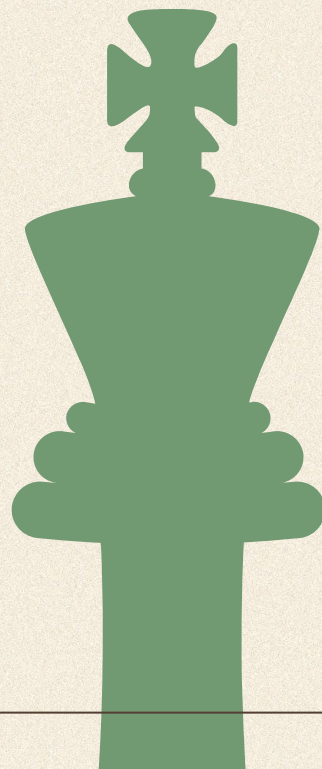
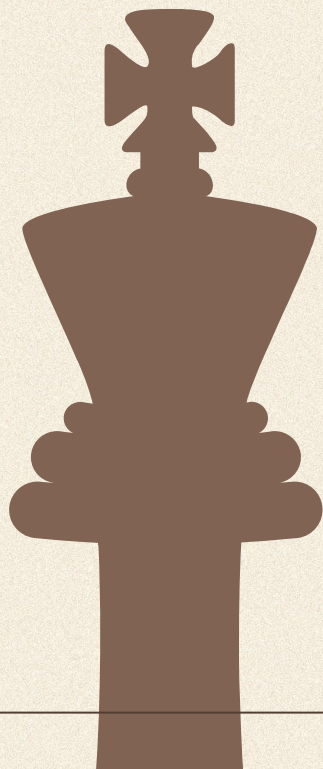


Joseph Hoane Chess Robot

Demo 1

Jordan Rios, Brayden Hayworth, Connor
Tynan, Evan Litzer

IOWA STATE
UNIVERSITY





The Problem

- Discovered vision module had incorrect implementation for the final step of detection
- There are some considerable challenges with board distortion as you get further from the camera
- Differences in piece size make capturing the entire piece challenging when it's a large piece like the King and Queen. When capturing big enough to get the largest pieces in whole cause you to have lots of background noise when capturing that same amount on a tile containing a pawn
- MuJoCo simulation module could only execute one command

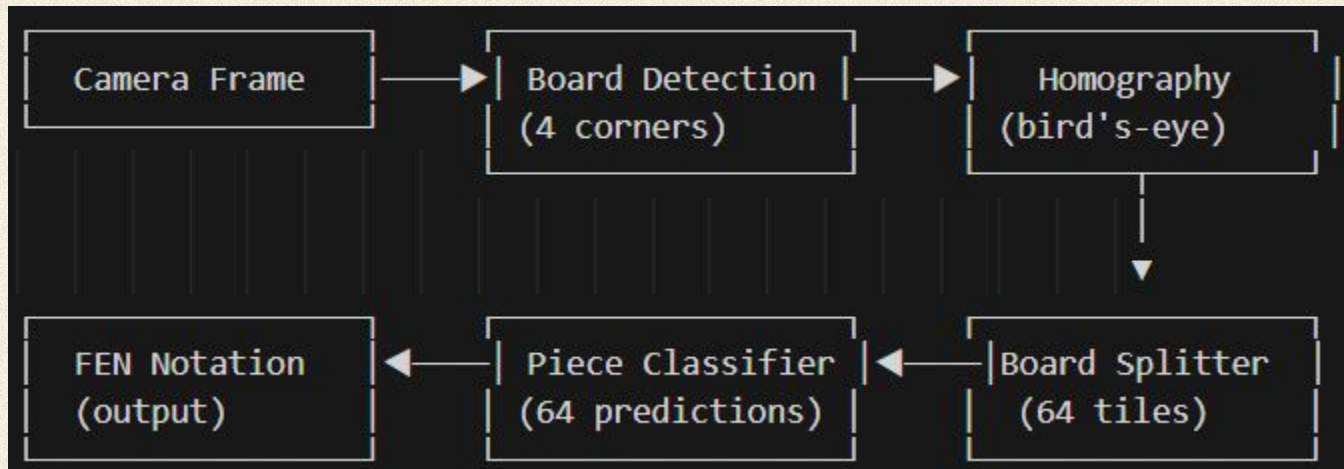


Vision Module

Current changes made to Vision Module:

- Went from using a Yolo-v8 model to a pre trained model (currently efficientnet_b0) and are using transfer learning via PyTorch to train our piece classifier.
- Changed the camera resolution to 1920x1080 (was 1280x720) and tile capture size to 128x128 (was 64x64).
- Created a dynamic tile cropping to combat the distortion caused by the camera angle.
- Created a training image capture system and auto labeling after providing a FEN notation of the board which you captured the data for.
- Removed some of the old code that wasn't being used anymore (Still more needs removed).
- Created a toggle for a feature that was capturing 7 images every 3 seconds that the program is running.

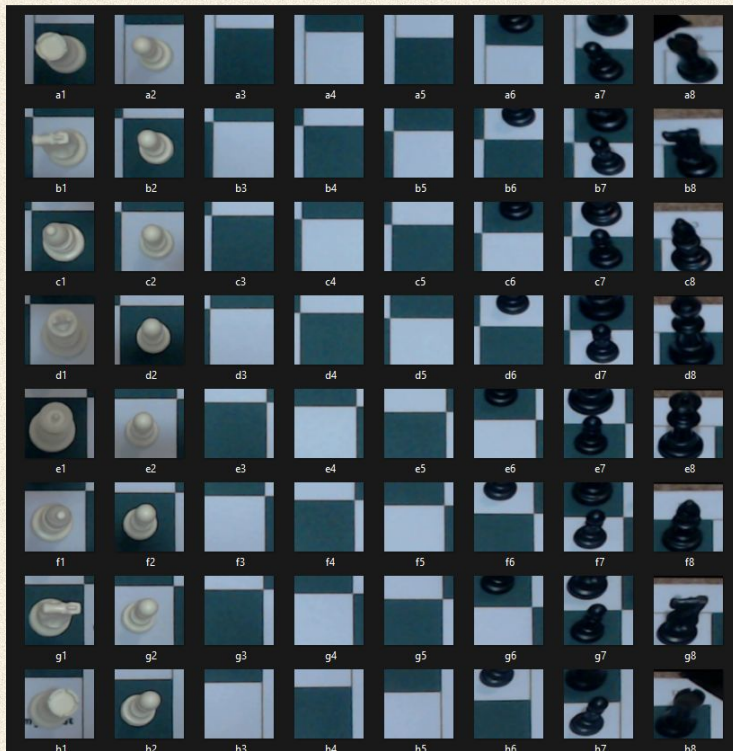
Vision Module New Flow



Vision Module (Calibration)



Vision Module (Capture Training data)



- Run the program when calibrated and set up a board position and press t to capture the board state.
- After Capture record the FEN in the FEN.txt file generated
- To auto label copy directory name containing the images and FEN into this command:
python
data/chess_dataset/tools/organize_captured_tiles.py [Dir]
[FEN]

Vision Module (Training Model)

```
Organized 64 files into C:\Development\SD\ug_hf_1\Backend\vision_module\data\chess_dataset\square_dataset\train
Run train_transfer_learning.py to train on the dataset.
PS C:\Development\SD\ug_hf_1\Backend\vision_module> python data\chess_dataset/tools/train_transfer_learning.py --epochs 25
Train: 52 samples, Val: 12 samples (88/20 split from train/)
Classes: ['B', 'K', 'N', 'P', 'Q', 'R', 'b_b', 'empty', 'k_b', 'n_b', 'p_b', 'q_b', 'r_b']
C:\Users\15157\AppData\Roaming\Python\Python312\site-packages\torch\utils\data\data_loader.py:668: UserWarning: 'pin_memory'
  found, then device pinned memory won't be used.
  warnings.warn(warn_msg)
Epoch 1/25   loss=2.4376   val_acc=0.5833
             -> Saved best model to C:\Development\SD\ug_hf_1\Backend\vision_module\Model\piece_classifier.pt
Epoch 2/25   loss=1.5967   val_acc=0.6667
             -> Saved best model to C:\Development\SD\ug_hf_1\Backend\vision_module\Model\piece_classifier.pt
Epoch 3/25   loss=0.7854   val_acc=0.6667
Epoch 4/25   loss=0.6192   val_acc=0.6667
Epoch 5/25   loss=0.2689   val_acc=0.6667
Epoch 6/25   loss=0.1653   val_acc=0.6667
Epoch 7/25   loss=0.1391   val_acc=0.7500
Epoch 8/25   loss=0.1278   val_acc=0.7500
Epoch 9/25   loss=0.0536   val_acc=0.7500
Epoch 10/25  loss=0.1139   val_acc=0.7500
Epoch 11/25  loss=0.0354   val_acc=0.7500
Epoch 12/25  loss=0.0352   val_acc=0.7500
Epoch 13/25  loss=0.0226   val_acc=0.7500
Epoch 14/25  loss=0.0537   val_acc=0.7500
Epoch 15/25  loss=0.0236   val_acc=0.7500
Epoch 16/25  loss=0.0060   val_acc=0.7500
Epoch 17/25  loss=0.0574   val_acc=0.7500
Epoch 18/25  loss=0.0125   val_acc=0.7500
Epoch 19/25  loss=0.0110   val_acc=0.7500
Epoch 20/25  loss=0.0221   val_acc=0.7500
Epoch 21/25  loss=0.0187   val_acc=0.7500
Epoch 22/25  loss=0.0079   val_acc=0.7500
Epoch 23/25  loss=0.0111   val_acc=0.7500
Epoch 24/25  loss=0.0263   val_acc=0.7500
Epoch 25/25  loss=0.0126   val_acc=0.7500

Training complete. Best val accuracy: 0.7500
Model saved to: C:\Development\SD\ug_hf_1\Backend\vision_module\Model\piece_classifier.pt
```

- Run the training python script:
python
data/chess_dataset/tools/train_transfer_learning.py
--[epochs]
- Select number of epochs you want to train for. The longer the better the accuracy the model can achieve.

Vision Module (Testing Piece Classifier)

```
Organized 64 files into C:\Development\SD\ug_hf_1\Backend\vision_module\data\chess_dataset\square_dataset\train
Run train_transfer_learning.py to train on the dataset.
PS C:\Development\SD\ug_hf_1\Backend\vision_module> python data/chess_dataset/tools/train_transfer_learning.py --epochs 25
Train: 52 samples, Val: 12 samples (88/20 split from train/)
Classes: ['B', 'K', 'N', 'P', 'Q', 'R', 'b_b', 'empty', 'k_b', 'n_b', 'p_b', 'q_b', 'r_b']
C:\Users\15157\AppData\Roaming\Python\Python312\site-packages\torch\utils\data\data_loader.py:668: UserWarning: 'pin_memory'
  found, then device pinned memory won't be used.
  warnings.warn(warn_msg)
Epoch 1/25   loss=2.4376   val_acc=0.5833
             -> Saved best model to C:\Development\SD\ug_hf_1\Backend\vision_module\Model\piece_classifier.pt
Epoch 2/25   loss=1.5967   val_acc=0.6667
             -> Saved best model to C:\Development\SD\ug_hf_1\Backend\vision_module\Model\piece_classifier.pt
Epoch 3/25   loss=0.7854   val_acc=0.6667
Epoch 4/25   loss=0.6192   val_acc=0.6667
Epoch 5/25   loss=0.2689   val_acc=0.6667
Epoch 6/25   loss=0.1653   val_acc=0.6667
Epoch 7/25   loss=0.1391   val_acc=0.7500
Epoch 8/25   loss=0.1278   val_acc=0.7500
Epoch 9/25   loss=0.0536   val_acc=0.7500
Epoch 10/25  loss=0.1139   val_acc=0.7500
Epoch 11/25  loss=0.0354   val_acc=0.7500
Epoch 12/25  loss=0.0352   val_acc=0.7500
Epoch 13/25  loss=0.0226   val_acc=0.7500
Epoch 14/25  loss=0.0537   val_acc=0.7500
Epoch 15/25  loss=0.0236   val_acc=0.7500
Epoch 16/25  loss=0.0060   val_acc=0.7500
Epoch 17/25  loss=0.0574   val_acc=0.7500
Epoch 18/25  loss=0.0125   val_acc=0.7500
Epoch 19/25  loss=0.0110   val_acc=0.7500
Epoch 20/25  loss=0.0221   val_acc=0.7500
Epoch 21/25  loss=0.0187   val_acc=0.7500
Epoch 22/25  loss=0.0079   val_acc=0.7500
Epoch 23/25  loss=0.0111   val_acc=0.7500
Epoch 24/25  loss=0.0263   val_acc=0.7500
Epoch 25/25  loss=0.0126   val_acc=0.7500

Training complete. Best val accuracy: 0.7500
Model saved to: C:\Development\SD\ug_hf_1\Backend\vision_module\Model\piece_classifier.pt
```

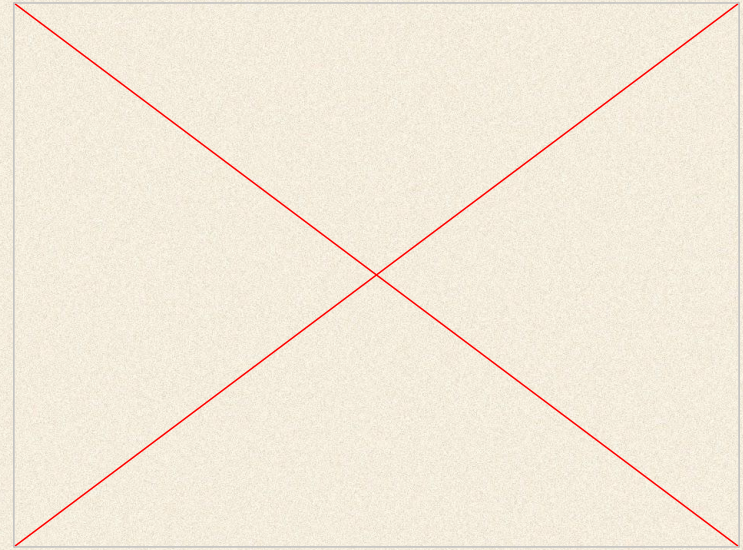
- Run the training python script:
python
data/chess_dataset/tools/train_transfer_learning.py
--[epochs]
- Select number of epochs you want to train for. The longer the better the accuracy the model can achieve.

Robotics: Motion Planner (Design)

- **Goal:** A class utility that takes move information as input and outputs command sequence
 - On initialization maps coordinates for center of each rank and file
 - Waits for input. On new input:
 - Parse move information:
 - Move: Position1 and Position2
 - Piece-to-Move (e.g. Pawn)
 - Conditions (e.g. Capture/Castle)
 - Determine positions in 3D space
 - Solves inverse kinematics
 - Sends joint angles/gripper state to robotic simulation

Robotics: Command Sequence

- Finding ways to implement issuing sequence of commands to robot
 - Utilizing a FIFO queue structure for command sequence
 - Command Flow (From “rest position”):
 - [Rest]
 - [Hover 1]
 - [Lower 1]
 - [Close Gripper]
 - [Hover 1]
 - [Hover 2]
 - [Lower 2]
 - [Open Gripper]
 - [Hover 2]
 - [Rest]





Robotics: Future Work

- Allow for real-time transmission of commands to simulation
- Implement motion planner using input from game engine
- Finish robotics pipeline



Full Game Engine Pipeline

1. **Vision FEN Move Detector**
 - a. Compares new FEN & previous FEN → outputs move made in UCI
2. **Board State**
 - a. Updates board object with move → current FEN and legal moves
3. **State Machine**
 - a. Takes current GameState enum and controls what happens next
4. **Stockfish**
 - a. FEN → Thinks with depth → Returns best move in UCI
5. **Move Classifier**
 - a. Move + current FEN → MoveInfo
6. **Robot Planner**
 - a. MoveInfo → Ordered list of robot steps

Board State & Fen Utilities

Board State: Crucial element in tracking game information

- Wraps python-chess library
- Tracks current position, full move and FEN history, current legal moves, and whose turn it is.
- All components read from BoardState object.

```
def __init__(self, fen=None, opponent_color="white"):
    self.board = chess.Board(fen or STARTING_FEN)
    self.move_history = []
    self.fen_history = []
    self.opponent_color = opponent_color
    self.robot_color = "black" if opponent_color == "white" else "white"
    self.last_known_turn = None
```

FEN Utils: FEN handling + validation

- Validate vision-sent FENs both structurally and semantically
 - Structural Ex: String format
 - Semantic Ex: Pawns on first rank
- Once vision is connected, will protect against vision module inconsistencies
- Turn, piece placement, compare position functions.

State Machine & Game Flow

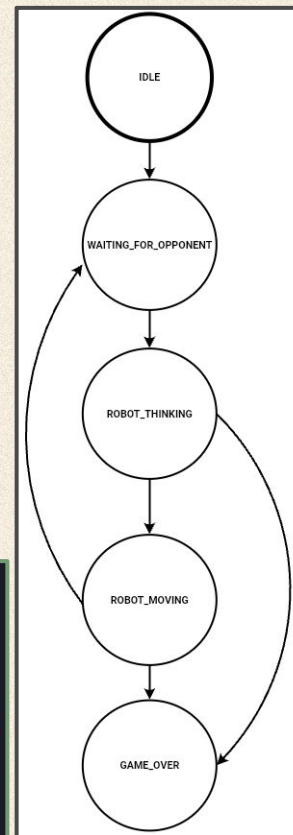
State machine controls turn-by-turn gameplay.

- IDLE, WAITING_FOR_OPPONENT, ROBOT_THINKING, ROBOT_MOVING, GAME_OVER.
- Game Over detector triggered → GAME_OVER state.
- No race conditions → cannot skip a state → valid transitions only

GameInfo object is the chess game's dashboard.

- Gives all components a readable object to understand game status.
- Includes state, FEN, turn, move #, last move made, pending move, Stockfish eval, check, and game result.
- Robot can read state, can be used for vision compare
- Can be used for debugging/logging in future

```
@dataclass
class GameInfo:
    state: GameState
    current_fen: str
    turn: str
    move_number: int
    last_move: Optional[str] = None
    pending_robot_move: Optional[str] = None
    evaluation: Optional[dict] = None
    is_check: bool = False
    game_result: Optional[str] = None
    message: str = ""
```



Game Engine Demo

```
[~venv] PS C:\Users\ivan\Desktop\CODE\PROJECT\backend\game_engine> python human_vs_robot.py
Enter your move (UCI format, e.g. e2e4): b5c3

You played: b5c3

rnbqkbnr
pppp.ppp
.....
...P...
...P...
...P...
pppp.ppp
R.BQKNNR

--- Game Info ---
state: waiting_for_opponent
current_fen: rnbqkbnr/pppp1ppp/8/4p3/PPPP1PPP/rnbqkbnr w KQkq - 1 2
turn: black
move_number: 2
last_move: b5c3
pending_robot_move: None
evaluation: None
is_check: False
game_result: None
message:

State: GameState.ROBOT_THINKING
Robot is thinking...

Stockfish plays: g6f6
Depth: 30
Score: -18 (cp)

Robot Action Plan:
Move type: MoveType.NORMAL
Move: e from g6 to f6

rnbqkbnr
pppp.ppp
.....
...P...
...P...
...P...
pppp.ppp
R.BQKNNR

State: GameState.ROBOT_THINKING
State: GameState.WAITING_FOR_OPPONENT

--- Game Info ---
state: waiting_for_opponent
current_fen: rnbqkbnr/pppp1ppp/8/4p3/PPPP1PPP/rnbqkbnr w KQkq - 0 2
turn: white
move_number: 2
last_move: g6f6
pending_robot_move: None
evaluation: {'best_move': 'g6f6', 'ponder_move': 'g1f3', 'depth': 30, 'nodes': 3221843, 'time_ms': 2000, 'score_type': 'cp', 'score_value': -10}
is_check: False
game_result: None
message:

Move history:
1. e2e4
...d5e5
2. b5c3
...g6f6

---
Turn: WHITE
State: GameState.WAITING_FOR_OPPONENT
Enter your move (UCI format, e.g. d2d4):
Interrupted. Shutting down...

Stockfish engine stopped.
Game complete.
[~venv] PS C:\Users\ivan\Desktop\CODE\PROJECT\backend\game_engine>
```

Game Over Detection

GAME OVER: Checkmate --> White wins.
Result: 1-0
Total moves: 41

Move Type & Robot Action

Move type: MoveType.NORMAL
Robot Action: Move Q from e6 to f7

Move, Depth, Engine Score

White plays: d6f7
Depth: 22
Score: 1087 (cp)

White plays: e6f7
Depth: 245
Score: 1 (mate)

```
--- Game Info ---
state: waiting_for_opponent
current_fen: rnbqkbnr/pppp1ppp/8/4p3/PPPP1PPP/rnbqkbnr w KQkq - 0 2
turn: white
move_number: 1
last_move: e7e5
pending_robot_move: None
evaluation: {'best_move': 'e7e5', 'ponder_move': 'g1f3', 'depth': 29, 'no
is_check: False
game_result: None
message:
```

Game Info



Game Engine Challenges

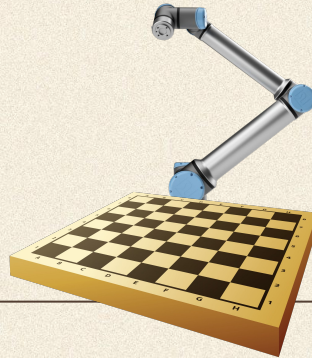
- **Vision module FEN consistency**
 - Even the best trained vision modules make mistakes.
 - Relying on move/FEN history for validation?
- **Move detection stabilization**
 - Time delay before state transition?
- **Illegal state recovery**
 - Robot makes mistake, wrong state is entered, etc.
- **Tracking castling rights**
- **Promotion move planning**
- **Draw detection besides stalemate**
 - 50/75 move rule, fivefold/threefold repetition, insufficient material

Future Plans

Client Goal: Get simulator working with picking up and placing pieces → get into the lab by Spring Break

Camera → FEN → Game Engine → MoveInfo → Move Planner → MuJoCo

- Need to connect all three parts of pipeline and have end-to-end handoff
- Inverse kinematics with planning movement in MuJoCo
- FEN stability with vision module and game engine + vision accuracy
- Picking up and placing piece movements → compatible with pieces



The image features a collection of chess pieces on a light beige background. In the center is a large, dark brown king piece with a cross on its crown. To its left is a dark brown rook. To its right is a green queen piece with a crown. Above the king is a green pawn. Below the king is a green rook. The entire scene is framed by a thin dark brown border with diamond-shaped corner pieces.

Questions?